

Référence technique

TDS Toolkit

Écrire un décodeur de protocole

Pour les oscilloscopes numériques des séries TDS 500, TDS 600 et TDS 700

Document TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Licence MIT.

TDSTK-DEC-01

Copyright © 2026 Jared Cabot. Tous droits réservés.

Ce document décrit l'interface des modules d'extension de décodage de TDS Toolkit. Les informations de cette publication remplacent celles de tout document publié antérieurement.

TDS Toolkit est un logiciel hôte ; ce n'est pas un produit Tektronix. Tektronix et TDS ne servent qu'à nommer les instruments pris en charge par le logiciel.

Ce logiciel est publié sous la licence MIT. LE LOGICIEL EST FOURNI « EN L'ÉTAT », SANS GARANTIE D'AUCUNE SORTE, EXPRESSE OU IMPLICITE. Le texte complet accompagne le programme sous le nom LICENSE.

Table des matières

Manuels associés	iv
1 Le modèle de décodeur	1-1
Ce qu'est un décodeur	1-1
Un fichier, un protocole	1-1
Le contrat	1-1
Découverte	1-1
Où résident les fichiers	1-1
2 Le Signal	2-1
Ce qui est fourni à un décodage	2-1
Échantillons et temporisation	2-1
Niveaux, pas échantillons	2-1
D'où vient un Signal	2-2
3 Le contrat du décodeur	3-1
Attributs de classe	3-1
Les quatre attributs	3-1
Rôles et sources	3-1
Les méthodes	3-2
decode	3-2
Paramètres	3-3
Le type Param	3-3
4 Frames et rendu	4-1
Le Frame	4-1
Un événement décodé	4-1
Statut	4-1
Rendu du résultat	4-2
Quatre petites méthodes	4-2
Couleur	4-2

5 Construire un décodeur	5-1
Un exemple détaillé	5-1
Le protocole	5-1
Les étapes	5-1
Le code	5-1
Imports et attributs	5-1
Récupérer les bits	5-2
Regrouper les mots	5-2
Rendu	5-3
Le raccourci série	5-4
Ne réimplémentez pas l'échantillonnage des bits	5-4
6 Installation et test	6-1
Installation	6-1
Déposer et ouvrir	6-1
Test	6-1
Le piloter avec un Signal synthétique	6-1
Testez chaque décodeur	6-2
Liste de contrôle	6-3
Avant de vous y fier	6-3

Liste des tableaux

Tableau 1–1: Fichiers du sous-système de décodage	1–2
Tableau 2–1: Membres d'un Signal	2–2
Tableau 3–1: Attributs de classe du décodeur	3–1
Tableau 3–2: Méthodes du décodeur	3–2
Tableau 3–3: Types de Param	3–3
Tableau 4–1: Champs de Frame	4–1
Tableau 4–2: Bits de statut de Frame	4–2

Préface

Ce manuel décrit comment ajouter un décodeur de protocole à TDS Toolkit. Un décodeur est un unique fichier Python. Il lit une forme d'onde capturée et renvoie une liste d'événements décodés ; le programme le répertorie, associe ses entrées, l'exécute et dessine le résultat sans aucun code supplémentaire.

Ce manuel s'adresse à un programmeur expérimenté, à l'aise avec Python. Aucune connaissance du reste du programme n'est nécessaire : toute l'interface tient en une classe de base et trois petits types de données, tous dans le module `tds_decode.py`.

Lisez ce manuel en regard de deux fichiers fournis avec le programme : `decoders/_template.py`, le squelette à copier, et `decoders/README.txt`, la version abrégée de ce qui suit.

Manuels associés

L'interface, et les fichiers à lire en regard de ce manuel :

- *decoders/_template.py* - le squelette à copier.
- *decoders/README.txt* - la version abrégée de ce qui suit.
- *tds_decode.py* - le moteur : la classe de base `Decoder` et les types `Signal`, `Frame` et `Param`.

Le modèle de décodeur

Ce qu'est un décodeur

	Un décodeur est un unique fichier Python, placé dans le dossier decoders, qui définit une sous-classe de Decoder. Chaque fichier correspond à un protocole. L'onglet Formes d'onde répertorie chaque décodeur qu'il trouve, de sorte qu'ajouter un protocole se résume à ajouter un fichier, sans rien changer d'autre dans le programme.
Un fichier, un protocole	Un décodage s'exécute entièrement sur une forme d'onde capturée que l'onglet Formes d'onde détient déjà : une voie lue depuis l'instrument, ou une référence stockée chargée depuis un fichier. Rien dans un décodeur ne touche à l'instrument. C'est là tout l'intérêt de la conception. Il fonctionne sur n'importe quel TDS, y compris les modèles de base dont le micrologiciel ne peut pas exécuter sa propre application de décodage de bus, car le décodage a lieu sur l'ordinateur hôte, sur des échantillons, après la capture. Toute l'interface se résume à la classe de base Decoder. Un décodeur définit quatre attributs de classe et implémente une méthode. Les trois autres méthodes ont des valeurs par défaut fonctionnelles, redéfinies uniquement pour rendre les champs propres au protocole dans le tableau et sur le graphique. Les trois types de données qu'un décodeur manipule, Signal, Frame et Param, sont décrits dans les chapitres qui suivent.
Le contrat	Au démarrage, et chaque fois que l'onglet Formes d'onde est ouvert, le programme parcourt le dossier decoders à la recherche de fichiers *.py, importe chacun isolément et instancie chaque sous-classe de Decoder qu'il trouve. Un fichier dont le nom commence par un tiret bas est traité comme un auxiliaire et ignoré, ce qui permet à _template.py de rester un modèle et de ne jamais apparaître comme un protocole vide dans la liste.
Découverte	L'importation est défensive. Un fichier qui ne s'importe pas, que ce soit à cause d'une erreur de syntaxe ou d'une mauvaise référence, est ignoré, avec une note écrite dans le journal du programme, plutôt que d'entraîner toute la liste avec lui. Un décodeur inachevé ne masque jamais ceux qui fonctionnent. Lorsque deux dossiers proposent un décodeur de même NAME, c'est le dossier examiné en dernier qui l'emporte, de sorte qu'une copie locale remplace celle fournie.

Où résident les fichiers

Les décodeurs fournis avec le programme résident dans un dossier `decoders` au sein de l'installation. Une copie accessible en écriture est conservée à côté du fichier de réglages, `tdstoolkit.json`, et est complétée à partir du jeu fourni à chaque exécution, exactement comme la bibliothèque de masques. Un décodeur ajouté dans une version ultérieure y est copié sans écraser un décodeur écrit localement ; un décodeur fourni supprimé réapparaît à l'exécution suivante, car le jeu fourni est censé être présent. Déposez un nouveau fichier dans ce dossier accessible en écriture et il apparaît dans la liste à la prochaine ouverture de l'onglet. Aucune reconstruction n'est nécessaire.

Tableau 1-1: Fichiers du sous-système de décodage

Fichier	Rôle
<code>tds_decode.py</code>	Le moteur : la classe de base <code>Decoder</code> , les types <code>Signal</code> , <code>Frame</code> et <code>Param</code> , les constantes de statut, l'auxiliaire série partagé et la découverte. Un décodeur l'importe et ne le modifie pas.
<code>decoders/_template.py</code>	Le squelette. Copiez-le pour commencer un nouveau décodeur.
<code>decoders/README.txt</code>	Les instructions abrégées et un aperçu des décodeurs d'exemple.
<code>decoders/.py</code>	Un protocole chacun : <code>uart.py</code> , <code>i2c.py</code> , <code>spi.py</code> , <code>can.py</code> et les autres. Lisez celui qui est le plus proche du protocole à ajouter.

Le Signal

Ce qui est fourni à un décodage

Échantillons et temporisation

La méthode `decode()` reçoit un Signal par entrée dont le protocole a besoin. Un Signal est un enregistrement capturé réduit à ce dont un décodeur a besoin : les échantillons entiers, et la temporisation qui convertit un indice d'échantillon en un temps. Il ne porte aucune tension ni aucune mise à l'échelle d'affichage. Un décodeur compare un échantillon à un seuil et n'a jamais besoin de connaître la tension, de sorte que les codes bruts du numériseur sont transmis sans mise à l'échelle et qu'un enregistrement d'un quart de million de points ne coûte rien à transmettre.

Le seuil de décision, la bande morte d'hystérésis et l'étendue du signal sont calculés avant l'appel de `decode()`. Un décodeur ne calcule pas de seuil propre. Le seuil est le milieu des extrêmes de l'enregistrement ; l'hystérésis vaut un huitième de l'étendue, assez large pour rejeter les oscillations de front sans empêcher un front lent de la franchir. Ces valeurs sont à lire, non à définir.

Niveaux, pas échantillons

Pour la plupart des protocoles, la forme utile de l'enregistrement n'est pas les échantillons bruts mais un niveau logique propre par échantillon. La méthode `digitize()` renvoie exactement cela : un 0 ou un 1 pour chaque échantillon, avec l'hystérésis propre à l'enregistrement appliquée afin que les oscillations de front ne se lisent pas comme plusieurs franchissements. Appelez-la une fois et parcourez le résultat. Un protocole qui a besoin des codes bruts, une ligne bipolaire à deux seuils par exemple, lit `sig.data` directement à la place.

Tableau 2-1: Membres d'un Signal

Membre	Signification
sig.data	Les échantillons entiers, une séquence indexable de codes bruts du numériseur.
sig.n	Le nombre d'échantillons, identique à len(sig.data).
sig.dt	Secondes par échantillon.
sig.t0	Le temps du premier échantillon, en secondes par rapport au déclenchement.
sig.first_index	Le décalage dans l'enregistrement du premier échantillon. Ajoutez-le à un indice d'échantillon local pour obtenir une position absolue dans l'enregistrement.
sig.threshold	Le niveau de décision, en codes. Défini par le framework.
sig.hysteresis	La bande morte autour du seuil, en codes. Fixée par le framework.
sig.span	La différence entre l'échantillon le plus haut et le plus bas de l'enregistrement. Définie par le framework ; une étendue proche de zéro signale une source plate, sans signal.
sig.time_at(i)	Le temps en secondes à l'indice d'échantillon local i, c'est-à-dire $t_0 + i \times dt$.
sig.digitize()	Un nouveau bytearray d'un niveau 0/1 par échantillon, avec hystérésis.
sig.auto_threshold()	Recalcule threshold, hysteresis et span. Déjà appelée par le framework ; à appeler uniquement sur un Signal construit à la main (voir la Section 6).

REMARQUE. Un Signal porte des codes bruts, pas des tensions. Comparez à sig.threshold, jamais à une tension. Un décodeur qui lit correctement à une échelle verticale lit alors correctement à toute autre échelle, car le seuil se déplace avec l'enregistrement.

D'où vient un Signal

Le Signal que decode() reçoit est construit par le programme, non par le décodeur, à partir de la forme d'onde capturée au moyen de la méthode de classe Signal.from_waveform(). Elle lit les niveaux entiers, l'intervalle d'échantillonnage et le décalage de déclenchement dans le préambule de la forme d'onde, construit le Signal et appelle auto_threshold() sur lui avant de le remettre au décodeur. Un Signal n'est construit à la main que lors de l'écriture d'un test, ce que couvre la Section 6.

Le contrat du décodeur

Attributs de classe

Une sous-classe de décodeur définit quatre attributs de classe. NAME et DESCRIPTION la nomment dans la liste des protocoles. SOURCES déclare les entrées de forme d'onde dont elle a besoin. PARAMS déclare les réglages qu'offre l'onglet. Tout le reste est hérité.

Tableau 3-1: Attributs de classe du décodeur

Attribut	Signification
NAME	Le nom du protocole. Affiché dans la liste et utilisé comme titre de l'étiquette du graphique. Laissez-le vide, ou « ? », et le chargeur ignore la classe ; c'est ainsi que le modèle reste hors de la liste.
DESCRIPTION	Une ligne pour la deuxième colonne de la liste.
SOURCES	Une liste de noms de rôle, dans l'ordre. Un seul rôle correspond à un protocole à un fil ; deux ou plus reçoivent chacun une source capturée qui leur est associée. Un « ? » final marque un rôle optionnel qui peut rester non associé.
PARAMS	Une liste d'objets Param, un contrôle chacun. Peut être vide.
COLOUR	La couleur du chevron pour un Frame sans erreur, au format « #rrggbb ». Un décodeur possède ses propres couleurs ; le programme n'a aucun réglage de couleur pour les décodeurs. Définissez-la pour donner une seule couleur à tout le protocole.
FAULT_COLOUR	La couleur du chevron pour un Frame en erreur. Rouge par défaut, afin qu'une erreur ressorte sur la couleur de n'importe quel protocole.

Rôles et sources

SOURCES est à la fois le nombre de formes d'onde dont un protocole a besoin et leurs noms. Un protocole à un fil déclare un seul rôle, par convention « Line ». Un protocole à deux fils tel que I2C déclare ["SDA", "SCL"] ; l'onglet affiche un sélecteur de source par rôle, et decode() reçoit un Signal pour chacun. Un point d'interrogation final, comme dans le ["CLK", "MOSI", "MISO?", "CS?"] de SPI, marque un rôle que l'opérateur peut laisser non associé ; son Signal est alors absent du dictionnaire que reçoit decode(), et le code de decode() vérifie sa présence.

Les méthodes

decode decode(self, signals, params) est l'unique méthode requise. signals est un dictionnaire associant chaque nom de rôle, sans son « ? » final éventuel, à son Signal ; un rôle optionnel non associé est absent. params est un dictionnaire associant chaque clé de Param à la valeur choisie par l'opérateur. La méthode renvoie un tuple à deux éléments : une liste d'objets Frame, et une chaîne de note d'une ligne.

La note est affichée au-dessus du tableau des résultats. Utilisez-la pour rapporter ce que le décodage a déduit, comme le débit binaire détecté, le nombre d'échantillons par bit et le nombre de Frames trouvés, ou, lorsque l'enregistrement ne peut pas être lu, pour en dire la raison. En cas d'échec, renvoyez une liste de Frames vide et une note qui explique le problème en une phrase.

Tableau 3-2: Méthodes du décodeur

Méthode	Renvoie
decode(signals, params)	(frames, note). Le décodage lui-même. Requise.
columns()	Les en-têtes du tableau après la colonne Temps, sous forme de liste de paires (heading, width_in_pixels). Redéfinissez-la pour nommer les champs du protocole.
row(frame, ascii)	Les cellules du tableau pour un Frame, correspondant à columns() après le temps. Redéfinissez-la pour formater les champs du protocole.
label(frame, ascii)	La chaîne utile la plus courte pour le chevron sur le graphique. Redéfinissez-la pour la garder concise.
colour(frame)	La couleur du chevron pour un Frame. Vaut par défaut COLOUR pour un Frame propre et FAULT_COLOUR pour un Frame en erreur. Redéfinissez-la pour colorer chaque chevron, selon frame.kind.

L'argument ascii passé à row() et label() est l'état de la bascule Hex / ASCII de l'onglet. Tenez-en compte lorsque c'est pertinent - un octet affiché comme un caractère ou comme deux chiffres hexadécimaux - et ignorez-le sinon.

Paramètres

Chaque Param de la liste PARAMS devient un contrôle sur l'onglet, de sorte qu'un protocole ajoute ses propres options sans aucun code d'interface utilisateur. Le constructeur est `Param(key, label, kind, choices, default, lo, hi, unit, note)`. `key` est la clé de dictionnaire que lit `decode()` ; `label` est la légende que voit l'opérateur ; `kind` est l'une de trois chaînes.

Tableau 3-3: Types de Param

kind	Contrôle et champs
"choice"	Une liste déroulante. <code>choices</code> est une liste de paires (<code>value</code> , <code>label</code>) ; la valeur est ce qui parvient à <code>params</code> , le <code>label</code> est ce qui est affiché. <code>default</code> est l'une des valeurs.
"int"	Une saisie d'entier. <code>lo</code> et <code>hi</code> la bornent ; <code>unit</code> est un suffixe de légende tel que « bits ». Une valeur <code>default</code> de 0 est par convention interprétée comme « auto », ce que la note devrait indiquer.
"bool"	Une case à cocher. <code>default</code> vaut <code>True</code> ou <code>False</code> .

Le type Param

Lorsque `default` est omis, il prend par défaut le premier choix pour un « choice », `False` pour un « bool » et 0 pour un « int ». La note optionnelle est affichée à côté du contrôle en guise d'indication.

Frames et rendu

Le Frame

Un événement décodé

Un Frame est un événement décodé : un caractère UART, un octet I2C, un mot SPI, une condition de bus. Le type est plat et partagé par tous les protocoles. Il porte une donnée principale, une donnée secondaire optionnelle, une longueur, un statut et un genre. Le sens des champs dépend du décodeur, puisque seul le décodeur qui a créé un Frame en assure le rendu, mais leur forme est fixe. Construisez un Frame avec des arguments nommés ; chaque champ a une valeur par défaut. Les deux champs d'indice sont les plus importants et les plus sujets à erreur. `index` et `end_index` sont des positions d'échantillon absolues dans l'enregistrement : le graphique dessine la boîte du chevron de l'un à l'autre. Formez-les comme `sig.first_index` plus l'indice d'échantillon local, exactement comme le temps provient de `sig.time_at()` de l'indice local.

Tableau 4-1: Champs de Frame

Champ	Signification
<code>time</code>	Le temps de début de l'événement en secondes, depuis <code>sig.time_at(local_index)</code> .
<code>index</code>	L'échantillon de début absolu : <code>sig.first_index</code> + début local.
<code>end_index</code>	L'échantillon de fin absolu : <code>sig.first_index</code> + fin locale. La boîte s'étend de <code>index</code> à <code>end_index</code> .
<code>value</code>	La donnée principale : l'octet, le mot, la valeur mesurée.
<code>tag</code>	Une donnée secondaire optionnelle : une adresse I2C, un mot MISO SPI, une fréquence mesurée. 0 si inutilisée.
<code>length</code>	La longueur de la donnée, dans l'unité que compte le protocole. Vaut 1 par défaut.
<code>status</code>	Un OU bit à bit des constantes de statut ; OK (zéro) lorsque le Frame est sans erreur.
<code>kind</code>	Une chaîne libre, telle que « byte », « addr », « data », « start » ou « stop », sur laquelle <code>row()</code> et <code>label()</code> aiguillent pour rendre chaque partie d'un Frame en tant que telle.

Statut status est un masque de bits. Un Frame propre est OK, c'est-à-dire zéro. Un Frame en erreur combine par OU une ou plusieurs des constantes du Tableau 4-2. Le framework dessine tout Frame dont le statut est non nul comme une erreur, un chevron rouge et un point d'exclamation en tête de l'étiquette, sans que le décodeur ait à faire quoi que ce soit de plus que positionner le bit.

Tableau 4-2: Bits de statut de Frame

Constante	Valeur	Signification
OK	0	Correct ; aucune erreur.
FRAMING_ERROR	1	Le Frame ne s'est pas terminé comme son format l'exige, comme un bit d'arrêt UART qui n'était pas une marque.
PARITY_ERROR	2	Un bit de parité était en désaccord avec les données.
TRUNCATED	4	L'enregistrement s'est terminé au milieu du Frame.
ACK_ERROR	8	Un octet qui attendait un acquittement n'a pas été acquitté.

Importez les constantes utilisées depuis `tds_decode` aux côtés de `Decoder`, `Frame` et `Param`. Ne définissez aucune valeur de statut au-delà de celles-ci ; le moteur de rendu ne comprend que les cinq ci-dessus.

Rendu du résultat

Quatre petites méthodes

La classe de base rend un Frame comme un simple octet : hexadécimal, ASCII et un mot de statut. Redéfinissez `columns()`, `row()` et `label()` pour employer plutôt les termes propres au protocole. `columns()` renvoie les en-têtes après la colonne Temps, chacun avec une largeur en pixels. `row()` renvoie une cellule par en-tête, dans le même ordre. `label()` renvoie la chaîne utile la plus courte pour le chevron dessiné sur la forme d'onde elle-même.

Un protocole dont les Frames portent des genres différents aiguille sur `frame.kind` dans ces méthodes, afin qu'un START, un octet d'adresse et un octet de données se lisent chacun comme tels dans le tableau et sur le graphique. Gardez `label()` concise : elle est dessinée à l'intérieur d'une petite boîte au-dessus de la trace, la bonne longueur est donc d'un à trois caractères.

Couleur La quatrième méthode, `colour(frame)`, définit la couleur du chevron, et le décodeur en a l'entière maîtrise : le programme n'a aucun réglage de couleur pour les décodeurs. Par défaut, elle peint un Frame propre avec la `COLOUR` de la classe et un Frame en erreur avec `FAULT_COLOUR`, de sorte que définir `COLOUR` donne une seule couleur à tout le protocole. Redéfinissez `colour()` et aiguillez sur `frame.kind` pour peindre une couleur par chevron, afin que l'adresse, les données et le tramage d'une même ligne se lisent chacun dans leur propre couleur. Renvoyez un « `#rrggbb` » ; retombez sur la classe de base pour les genres que le décodeur ne colore pas lui-même.

Construire un décodeur

Un exemple détaillé

Le protocole

Ce chapitre construit un décodeur complet à partir du modèle. Le protocole d'exemple est un unique fil auto-cadencé qui porte un bit par impulsion : la ligne monte, reste haute pendant une fraction de la période de bit, puis retombe ; un état haut court est un 0 et un état haut long est un 1, selon que le temps à l'état haut dépasse ou non la moitié de la période. Les bits sont regroupés en mots d'une taille et d'un ordre des bits choisis. C'est la famille de la ligne de pixels WS2812 et de la télécommande infrarouge NEC, et elle ne figure pas parmi les protocoles intégrés, c'est donc un véritable nouveau décodeur plutôt que la copie d'un décodeur existant.

Les étapes

1. *Copiez le modèle* : Copiez `decoders/_template.py` vers `decoders/pulsebits.py`.
2. *Nommez le protocole* : Renommez la classe et définissez `NAME`, `DESCRIPTION` et `SOURCES`.
3. *Déclarez les réglages* : Remplissez `PARAMS` avec les contrôles de taille de mot et d'ordre des bits.
4. *Écrivez le décodage* : Récupérez les bits depuis la piste de niveaux, puis regroupez-les en mots.
5. *Affichez les champs* : Redéfinissez `columns()`, `row()` et `label()` pour afficher le mot en hexadécimal et en binaire.
6. *Testez-le* : Pilotez le décodeur avec un Signal synthétique et vérifiez le résultat par assertion, comme le montre la Section 6.

Le code

Imports et attributs

L'en-tête importe uniquement depuis `tds_decode`, nomme la classe et déclare les deux entrées que l'onglet proposera : une taille de mot et un ordre des bits. `SOURCES` contient un seul rôle, donc l'onglet affiche un unique sélecteur de source.

```

from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]

```

Récupérer les bits

decode() prend l'unique Signal, se prémunit contre une source plate, et réduit l'enregistrement à un niveau 0/1 par échantillon avec digitize(). Elle parcourt ensuite la piste de niveaux. Chaque bit commence sur un front montant ; la série d'échantillons hauts est l'impulsion, la série d'échantillons bas qui suit se poursuit jusqu'au front montant suivant, et les deux réunies forment la période de bit. La comparaison du temps à l'état haut avec la période détermine la valeur du bit.

```

def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = [] # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]: # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1

```

Regrouper les mots

Les bits sont ensuite pris par groupes de `word_bits`, du plus ou du moins significatif d'abord selon le paramètre. Un groupe incomplet parce que l'enregistrement s'est terminé est tout de même émis, avec son statut marqué `TRUNCATED` afin que le moteur de rendu le dessine comme une erreur. Chaque Frame tire son temps du premier bit et s'étend du début du premier bit à la fin du dernier bit. La note indique les totaux.

```
frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))
```

Rendu

Les trois méthodes de rendu affichent le mot en hexadécimal et en binaire de sa propre largeur en bits, et signalent un mot tronqué. `label()` s'en tient à deux chiffres hexadécimaux avec un point d'exclamation en tête en cas d'erreur, ce qui est la convention que suivent les décodeurs intégrés.

```
def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "%0db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text
```

Ce fichier est un décodeur complet et répertoriable. Déposé dans le dossier `decoders`, il apparaît dans l'onglet sous le nom « Pulse-width bits » avec un champ de taille de mot et une liste déroulante d'ordre des bits, et décode toute source associée à son unique entrée.

Le raccourci série

**Ne réimplémentez pas
l'échantillonnage des bits**

L'exemple détaillé échantillonne lui-même les bits parce que son protocole est auto-cadencé et ne ressemble à rien d'intégré. Pour tout ce qui relève de la famille série asynchrone, comme RS-232, RS-422/485, MODBUS, MIDI, LIN, XBee et DMX512, ne réimplémentez pas l'échantillonnage des bits. Appelez l'auxiliaire partagé et testé sur corpus `serial_bytes()`, qui réalise la détection automatique du débit, l'échantillonnage au centre du bit, ainsi que les contrôles de parité et de bit d'arrêt de l'unique manière éprouvée, et construisez le tramage sur le flux d'octets qu'il renvoie.

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` renvoie quatre valeurs : les Frames d'octets, une chaîne d'erreur vide en cas de succès, le débit en bauds détecté et le nombre d'échantillons par bit. Une valeur baud de 0 lui demande de détecter le débit. `inverted` à `True` interprète une polarité repos-bas, actif-haut, comme celle du vrai RS-232. Un protocole de plus haut niveau tel que MODBUS ou LIN appelle `serial_bytes` pour les octets bruts, puis les regroupe et les contrôle dans ses propres Frames ; consultez `modbus.py` et `lin.py` pour ce schéma.

Installation et test

Installation

Déposer et ouvrir

Copiez le fichier terminé dans le dossier decoders accessible en écriture à côté de tdstoolkit.json et ouvrez, ou rouvrez, l'onglet Formes d'onde. Le décodeur apparaît dans la liste des protocoles. Il n'y a ni étape d'enregistrement ni reconstruction. Associez une source capturée à chacun de ses rôles, réglez ses paramètres et exécutez-le.

REMARQUE. Si le décodeur n'apparaît pas, c'est qu'il ne s'est pas importé. Recherchez dans le journal du programme une ligne nommant le fichier et l'exception. Le chargeur ignore un fichier qui ne s'importe pas plutôt que de faire échouer toute la liste, de sorte qu'un décodeur défectueux échoue silencieusement.

Test

Le piloter avec un Signal synthétique

Un décodeur est une fonction pure de ses Signals et de ses paramètres, il se teste donc sans instrument en construisant un Signal à partir d'un tableau d'échantillons fabriqué, en appelant decode() et en vérifiant le résultat par assertion. Construisez les échantillons de façon que leur signification soit connue, enveloppez-les dans un Signal, appelez auto_threshold(), l'unique étape que le framework effectue normalement, et vérifiez les Frames.

```

from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20          # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20        # idle low tail
    sig = Signal(stream, 1e-6)     # 1 us per sample
    sig.auto_threshold()           # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()

```

Testez chaque décodeur

Chaque décodeur intégré est vérifié de cette façon. Le décodeur RS-232 est exécuté sur un corpus de captures réelles ; tous les autres sont vérifiés à l'aide d'un vecteur synthétique dans `tdstest.py`, chacun avec un mutant correspondant dans `bitecheck.py` qui casse le décodeur à dessein et confirme que le test le remarque. Ajoutez un cas de test pour chaque protocole ajouté.



ATTENTION. *Un décodage non testé finira par dériver. Ajoutez un vecteur synthétique pour chaque décodeur ajouté, et un mutant qui prouve que le vecteur remplit réellement son rôle, avant de vous fier au résultat.*

Liste de contrôle

Avant de vous y fier

- Le fichier est nommé en minuscules, sans espaces, et ne commence pas par un tiret bas.
- La classe définit NAME, DESCRIPTION, SOURCES et PARAMS, et NAME n'est pas vide.
- decode() renvoie un tuple (frames, note) dans tous les chemins, y compris les chemins d'échec.
- Les index et end_index d'un Frame sont absolus : sig.first_index plus l'indice d'échantillon local.
- Un enregistrement illisible renvoie une liste vide et une note qui en explique la raison, jamais une exception.
- columns(), row() et label() s'accordent sur le nombre et l'ordre des champs.
- Un test synthétique pilote le décodeur et vérifie par assertion les valeurs décodées.